Software quality & Software standards

Measurements!

Software quality & Software standards: Measurements

- Reviews/Inspections & Testing
- Reliability Engineering
- *Software Project Management*

Review Techniques

Que:

- What is the main difference between Reviews/Inspections & Testing

Ans:

- Reviews/Inspections **(static)** & Testing **(Dynamic)**

What Are Reviews?

- A meeting conducted by technical people for technical people
- A technical assessment of a work product created during the software engineering process
- A software quality assurance mechanism

Class Discussion

- *Error*—vs.—*Defect*

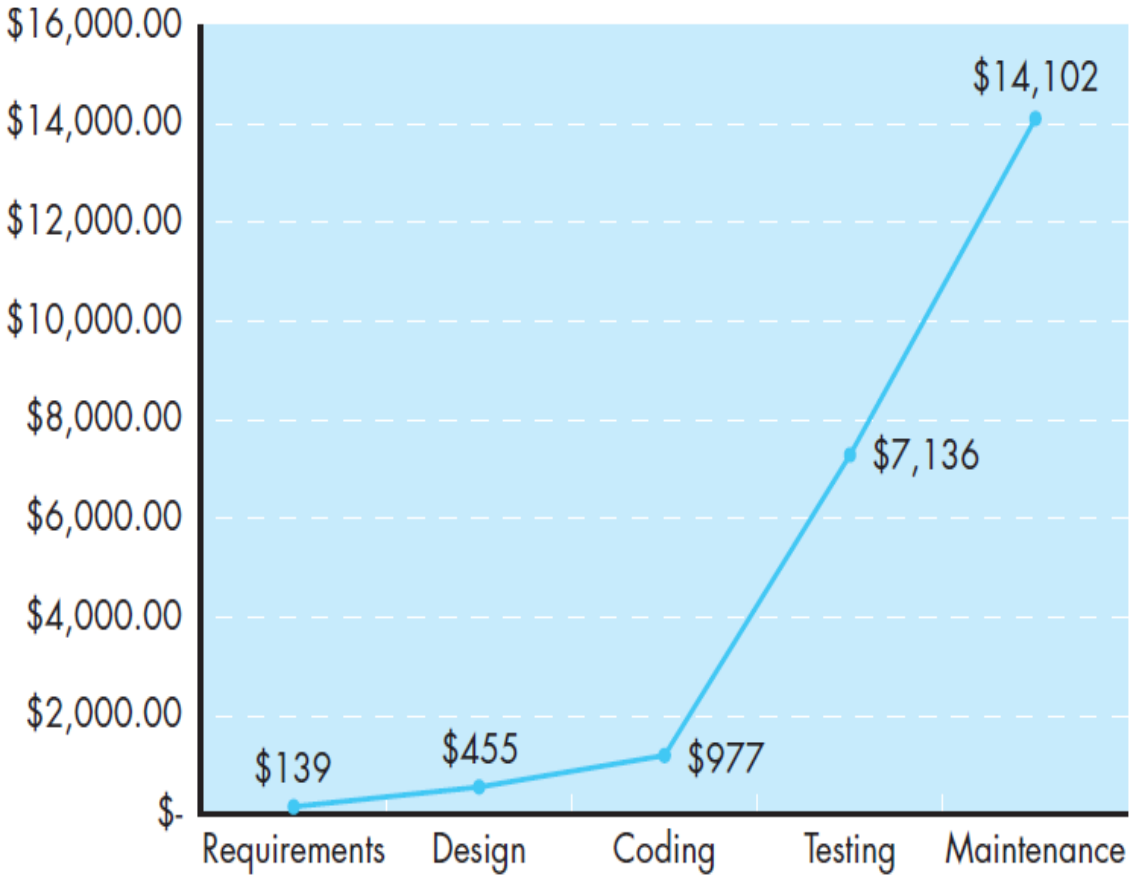
- *NB: errors and defects have very different economic, business, psychological, and human impact*

What Do We Look For?

- Errors and defects
  - *Error*—a quality problem found *before* the software is released to end users
  - *Defect*—a quality problem found only *after* the software has been released to end-users

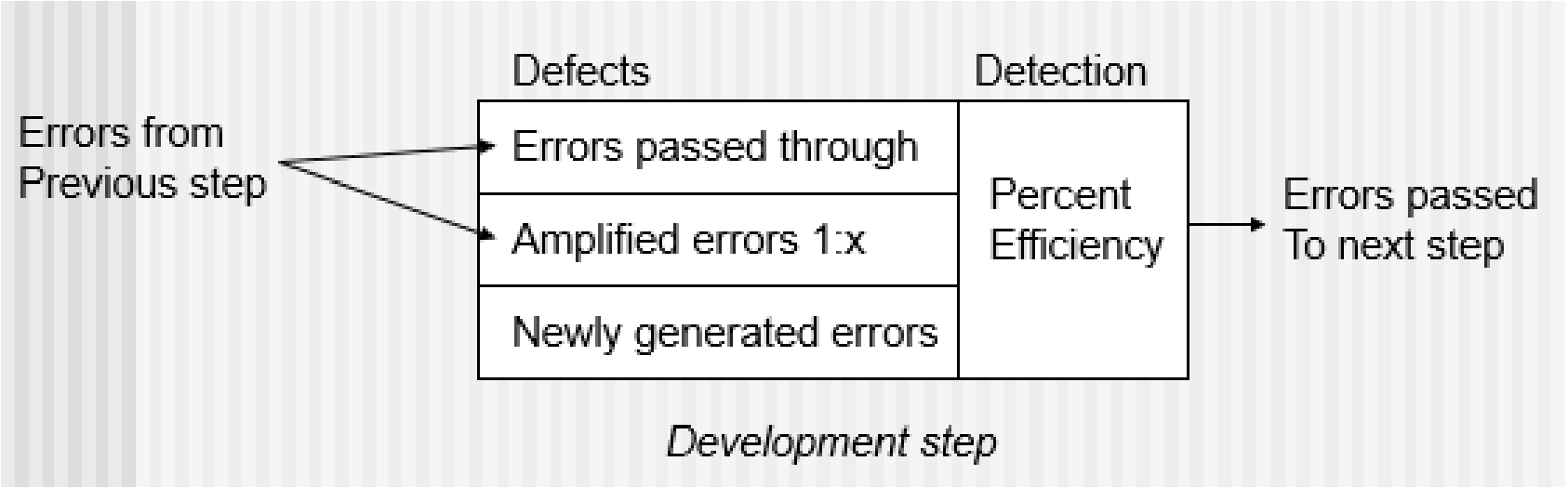
Relative cost of correcting errors and defects!

Relative cost of correcting errors and defects
Source: Adapted from [Boe01b].



Defect Amplification

- **A *defect amplification model*** [IBM81] can be used to illustrate the **generation** and **detection** of errors during the design and code generation actions of a software process.



Software Reliability Calculation

Scenario:
Assume that 10 errors have been introduced in the **requirements** model and that each error will be amplified by a factor of 2:1 into **design** and an addition 20 design errors are introduced and then amplified 1.5:1 into **code** where an additional 30 errors are introduced. Assume further that all **unit testing** will find 30 percent of all errors, **integration** will find 30 percent of the remaining errors, and **validation tests** will find 50 percent of the remaining errors. (No reviews are conducted).
Reconsider the situation described in the above scenario, but now assume that **requirements, design, and code reviews** are conducted and are in uncovering all errors at that step 60 percent effective

Software Reliability Calculation

Solutions:

Calculate Total Errors After Amplification:

From Requirements to Design:

Amplified errors: 10 req. errors * 2 = **20 errors**

Plus, new design errors: +20

Total Design Errors: 20 + 20 = **40 errors**

From Design to Code:

Amplified errors: 40 design errors *1.5 = **60 errors**

Plus, new code errors: +30

Total Code Errors (before testing): 60 + 30 = **90 errors**

Software Reliability Calculation

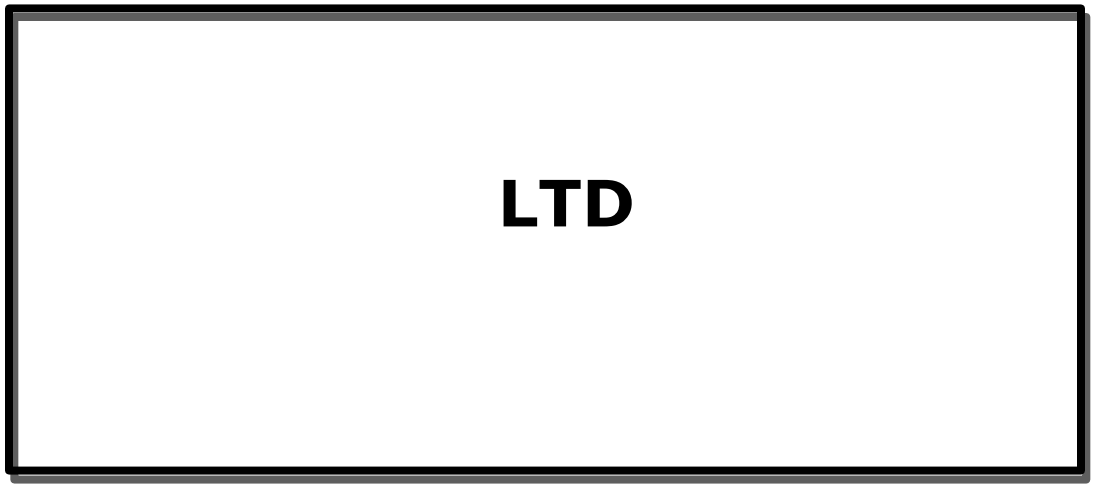
Testing:
Unit testing finds 30% of errors: $0.3 \times 90 = \mathbf{27}$ **errors** found. Remaining errors: $90 - 27 = \mathbf{63}$ **errors**.

Integration testing finds 30% of remaining errors: $0.3 \times 63 = \mathbf{18.9}$ **errors**.
Remaining errors: $63 - 18.9 = \boldsymbol{44.1\ errors}$

Validation testing finds 50% of remaining errors: $0.5 \times 44.1 = \mathbf{22.05}$ **errors**.
Remaining errors: $44.1 - 22.05 = \mathbf{22.05}$ **errors**

Errors remaining after all testing: 22.05 errors

— Tutorials —



Error in the process Software Reliability— **Tutorial**

- Same question but assume that requirements, design, and code reviews are conducted and are 34 percent effective in uncovering all errors at that step. How many errors will be released to the field?

Error in the process Software Reliability— **Tutorial**

- Suppose a software project initially contains 15 errors in its requirements model. Each error will propagate with a factor of 3:1 into the design phase, resulting in an additional 45 design errors. These design errors will then be magnified by a factor of 1.8:1 into the code, introducing an extra 81 errors. Assume that during unit testing, 40 percent of all errors are detected, followed by integration testing uncovering 25 percent of the remaining errors, and finally, validation tests finding 60 percent of the remaining errors. If no reviews are conducted, how many errors will persist and be released into the field?

Reliability Engineering

Software Reliability Calculation

Software Reliability Calculation

- A simple measure of reliability is **mean-time-between-failure (MTBF)**, where

  MTBF = MTTF + MTTR

- The acronyms
- **MTTF**: **mean-time-to-failure**
- **MTTR**: **mean-time-to-repair**

Software Reliability Calculation

- **Software availability** is the probability that a program is operating according to requirements at a given point in time and is defined as

- **Availability = [MTTF/(MTTF + MTTR)] x 100%**

- The acronyms
- **MTTF**: **mean-time-to-failure**
- **MTTR**: **mean-time-to-repair**

Software Reliability Calculation

Scenario:
GlobalBank operates an online transaction processing system that supports millions of customers worldwide. The system handles ATM withdrawals, credit card transactions, and mobile payments. Due to its critical financial nature, regulators require banks to maintain a minimum availability of **99.5%**.
The system's engineers conducted an operational reliability study and observed the following:
The average **Mean Time To Failure (MTTF)** is **400 hours** (after 400 hours of continuous operation, the system is likely to fail).
The average **Mean Time To Repair (MTTR)** is **10 hours** (time taken to restore the system after a failure).
The bank is considering investing in additional **redundancy** measures (active–passive replication), which could reduce MTTR to **2 hours**, though at a significant infrastructure cost.

Software Reliability Calculation

Questions:

(a) Using the given MTTF and MTTR values, calculate the system's current **availability percentage**.

(b) Based on your calculation, evaluate whether the system meets the regulator's minimum requirement of 99.5% availability.

(c) If redundancy reduces MTTR to 2 hours, recalculate the system availability.

Would this investment justify the cost from a regulatory and business continuity perspective? Provide a critical discussion.

Software Reliability Calculation

Solutions:

(a) Availability Calculation:

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

$$Availability = \frac{400}{400 + 10} = \frac{400}{410} \approx 0.9756 = 97.56\%$$

Software Reliability Calculation

Solutions:

(b) Evaluation:
The required availability is 99.5%. The system achieves only 97.56% → **Fails to meet regulatory requirements**.

(c) With redundancy (MTTR = 2 hours):

$$Availability = \frac{400}{400 + 2} = \frac{400}{402} \approx 0.995 = 99.5\%$$

This exactly meets the regulator's threshold.

Tutorial

Case Study Scenario: ABC Software Solutions is a leading provider of cloud-based accounting software for small businesses. One of their flagship products, "FinanceMaster," is widely used by entrepreneurs and business owners to manage their financial transactions, generate reports, and streamline accounting processes.

- The reliability and availability of FinanceMaster are critical factors in ensuring customer satisfaction and business success. To assess the software's performance, ABC Software Solutions utilizes the mean-time-between-failure (MTBF) metric, which is calculated as the sum of the mean-time-to-failure (MTTF) and the mean-time-to-repair (MTTR).
- MTTF represents the average time between software failures, while MTTR denotes the average time required to repair the software after a failure occurs.
- Given the importance of software availability in maintaining customer trust and confidence, ABC Software Solutions also measures the software's availability, defined as the probability that FinanceMaster is operating according to requirements at any given point in time. This availability metric is crucial for ensuring uninterrupted access to critical financial data and functionalities.
- Let's consider a scenario where FinanceMaster has the following reliability metrics:
- Mean-Time-to-Failure (MTTF) = 200 hours
- Mean-Time-to-Repair (MTTR) = 20 hours
- Your task is to calculate the availability of FinanceMaster and analyze the implications of this metric for ABC Software Solutions' business operations and customer satisfaction. Additionally, discuss potential strategies for improving software availability and mitigating the impact of system failures on users.

Reliability Prediction Using Exponential Model — **Tutorial**

Scenario:

A medical monitoring device has a failure rate (λ) of 0.0002 failures/hour.

(a) Calculate the probability that the device operates without failure for 1000 hours.

(c) If regulatory standards require 85% reliability at 1000 hours, does the device pass?

Software Project Management

Three-point estimate technique

Three-point estimate technique

- The three-point estimate technique combines optimistic, pessimistic, and most likely estimates to determine a realistic **timeframe for a project**:

1. **Optimistic estimate (O)**
2. **Most likely estimate (M)**
3. **Pessimistic estimate (P)**

Three-point estimate technique

- TechSolutions Inc. has been contracted to develop a mobile app for a local fitness center. The project manager uses the three-point estimate technique to assess the timeline for the development phase. The most likely estimate for completing the app development is 8 weeks, but unforeseen technical challenges could extend it to 12 weeks. Conversely, if everything goes smoothly, the development might be finished in just 6 weeks. Based on the three-point estimate:

1. what is the most realistic timeframe TechSolutions Inc. should communicate to the fitness center for completing the mobile app development?

2. What proactive measures can TechSolutions Inc. implement to mitigate the risk of unforeseen technical challenges and ensure timely completion of the mobile app development project?

Three-point estimate technique

- Solution: The three-point estimate technique combines optimistic, pessimistic, and most likely estimates to determine a realistic timeframe for a project.
- *Given:*
- *Optimistic estimate (O) = 6 weeks*
- *Most likely estimate (M) = 8 weeks*
- *Pessimistic estimate (P) = 12 weeks*

Three-point estimate technique

- *Expected Timeframe: (O + 4M + P) / 6*
- ***Weighted Average (emphasizing Most Likely)**: (O + 4M + P) / 6*
- *= (6 + 32 + 12) / 6*
- *= 50 / 6*
- *= **8.33 weeks***
- *It's important to note that this is an estimate and actual times may vary based on project execution. It's also good practice to include a buffer in the timeline to account for any unforeseen circumstances.*

Software Project Management

Cost Analysis and Resource Management in Software Projects

Cost Analysis and Resource Management in Software Projects

- Sunshine Software is developing a new cloud-based project management tool. Initial estimates suggest the project will require 8 developers for a period of 9 months. The average developer salary at Sunshine Software is R120,000 per year (including benefits). There are also additional overhead costs estimated at 25% of the total salaries.

1. What is the total cost of developer salaries and overhead for the project?

2. What strategies can Sunshine Software implement to optimize resource utilization and minimize costs during the development of the cloud-based project management tool?

Cost Analysis and Resource Management in Software Projects

Solution!

- To calculate the total projected cost for the development team, we need to consider both the **salaries** and the **overhead costs**.

Given:

- *Number of developers: 8*
- *Project duration: 9 months*
- *Average developer salary: R120,000 per year*

Cost Analysis and Resource Management in Software Projects

Solution Cont.!

- **Step 1**: **Monthly Developer Salary** = [Annual Salary / 12)]
  =[R120,000 × 12]
  =R10,000

- **Step 2: Total Salary:**

Total Salary = [Salary × # of Developer × 9 months]
=[R10,000 × 8 × 9]
=R720,000

Cost Analysis and Resource Management in Software Projects

Solution Cont.!

- **Step 3: Overhead Costs**:

Overhead cost = [Total Salary × Overhead Rate]
= R720,000 × 0.25
=R180,000

- **Step 4: Total Projected Cost:**

Total Projected cost = [Total Salary cost + Overhead Cost]
= R720,000 + R180,000
=R900,000

Therefore, the total cost of developer salaries and overhead for the project is R900,000.

Software Metrics and Project Estimation

Lines of code (LOC)-based estimation technique

Lines of code (LOC)-based estimation technique

- You are leading a project for a software development company tasked with creating a mobile application for a local restaurant. This application will allow customers to browse the menu, place orders for pickup or delivery, make reservations, and provide feedback on their dining experience. The total estimated lines of code (LOC) for the application is 9,500.

1. As part of your project planning, you need to estimate the effort and cost required to build this software using a lines of code (LOC)-based estimation technique. Your organization's productivity rate is 500 LOC per person-month, and the labor rate, including overhead costs, is R6,000 per person-month.
2. Discuss potential factors or risks that could impact the accuracy of the lines of code (LOC)-based estimation technique in predicting the effort and cost of software development projects. How can project managers mitigate these factors to improve estimation accuracy?

Lines of code (LOC)-based estimation technique

- Solution:
- To estimate the **effort** and **cost** required to build the mobile application for the local restaurant, we'll use the given lines of code (LOC)-based estimation technique.

Given:

- *Total estimated lines of code (LOC):* ***9,500***
- *Productivity rate:* ***500 LOC*** *per person-month*
- *Labor rate, including overhead costs:* ***R6,000*** *per person-month*

Lines of code (LOC)-based estimation technique

Solution!

- Step 1: Calculate Effort Required:

Effort (in person-months) = [***Total LOC / Productivity rate***]

Effort = 9,500 LOC / 500 LOC per person-month

= **19 person-months**

- Step 2: Calculate Cost of Development:

Cost of development = **Effort × Labor rate**

Cost = 19 person-months × R6,000 per person-month

= **R114,000**

Therefore, the effort required to build the software is approximately **19 person-months**, and the cost of development is **R114,000**

Further Reading!

Prescribed Textbooks:

(a) Software Engineering, by Ian Sommerville 10th edition, Pearson Education Limited, 2016.

(b) Software Engineering – A Practitioner's Approach 8th edition Roger Pressman McGraw Hill Education (2015).